3.5 Multicollinearity

- In multiple linear regression, with many explanatory variables $\vec{x} = (x_1, \ldots, x_d)$ in the model to explain the response variable y, there is a higher chance that some of these d explanatory variable are correlated. In this case, $\vec{x} \in \mathbb{R}^d$ might not represent d separate pieces of information i.e. there is some redundancy. This is called **multicollinearity**. (Note this concept is focused on the explanatory variables $x_1, \ldots, x_d$ themselves and has nothing to do with the response variable y.)
- If two explanatory variables x_1 and x_2 are perfectly correlated i.e. $\mathrm{Corr}(x_1, x_2) = 1$, then their roles are exchangeable. For example, if $x_1 = x$ and $x_2 = 4\,x$, then the model equation

$$y = 4\,x_1 + 0\,x_2 = 0\,x_1 + 1\,x_2 = 2\,x_1 + 0.5\,x_2$$

  can be represented in (infinitely) many ways. The model is **non-identifiable** because of the non-uniqueness of the model parameters. (This is exactly the same as the concept of linear dependence of vectors in linear algebra.)

```
x1 <- c(0.4, 0.8, 1.2, 1.7, 2.1)
y <- c(2, 5, 4.3, 4.2, 8)

x2 <- 2 * x1 # Corr(x1, x2) = 1
summary(lm(y ~ x1 + x2)) # Same as y ~ x1 + I(x1 * 2)   See ?formula
```

```

Call:
lm(formula = y ~ x1 + x2)

Residuals:
      1       2       3       4       5
-0.5410  1.4309 -0.2972 -1.6823  1.0896

Coefficients: (1 not defined because of singularities)
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    1.513      1.488   1.017   0.3841
x1             2.570      1.077   2.386   0.0971 .
x2                NA         NA      NA       NA
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.466 on 3 degrees of freedom
Multiple R-squared:  0.6549,    Adjusted R-squared:  0.5399
F-statistic: 5.694 on 1 and 3 DF,  p-value: 0.09707
```

- Multicollinearity makes numerical computation unstable. Parameter estimation becomes very sensitive i.e. a small change in an explanatory variable x_i might result in a large change in the values of (more than one) model parameters. Consequently, the model has poor **explainability** because a $+1$ unit increase in an explanatory variable x_i does not imply a $+\beta_i$ unit increase in the response variable y anymore. (Part of the reason is because a change in x_i will also imply a change in other highly correlated explanatory variables so the scenario of "$+1$ unit in x_i while having all else being the same" does not hold.)

```
set.seed(1)

x1 <- c(0.4, 0.8, 1.2, 1.7, 2.1)
y <- c(2, 5, 4.3, 4.2, 8)

x2 <- x1 + rnorm(length(x1), mean = 0, sd=1e-4) # Add small noise
x2
```

```
[1] 0.3999374 0.8000184 1.1999164 1.7001595 2.1000330
```

```
1 - cor(x1, x2) # Corr(x1, x2) is almost 1
```

```
[1] 6.710243e-09
```

```
summary(lm(y ~ x1 + x2)) # Large magnitude of parameters
```

```

Call:
lm(formula = y ~ x1 + x2)

Residuals:
      1       2       3       4       5
-0.5868  1.6740 -0.8504 -1.0331  0.7963

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)     0.9954     1.9294   0.516    0.657
x1           5935.0921 10586.4781   0.561    0.631
x2          -5932.0427 10585.6229  -0.560    0.632

Residual standard error: 1.669 on 2 degrees of freedom
Multiple R-squared:  0.7018,    Adjusted R-squared:  0.4035
F-statistic: 2.353 on 2 and 2 DF,  p-value: 0.2982
```

```
x3 <- x2
x3[2] <- x3[2] + 1e-5 # Small change in explanatory variable
x3
```

```
[1] 0.3999374 0.8000284 1.1999164 1.7001595 2.1000330
```

```
summary(lm(y ~ x1 + x3)) # Parameters become sensitive
```

```
Call:
lm(formula = y ~ x1 + x3)

Residuals:
      1       2       3       4       5
-0.6016  1.6788 -0.7903 -1.1210  0.8341

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)     1.087      1.929   0.564    0.630
x1           5174.437  10574.416   0.489    0.673
x3          -5171.461  10573.587  -0.489    0.673

Residual standard error: 1.697 on 2 degrees of freedom
Multiple R-squared:  0.6918,    Adjusted R-squared:  0.3836
F-statistic: 2.245 on 2 and 2 DF,  p-value: 0.3082
```

- In general, multicollinearity can occur when one explanatory variable is correlated with a mix of a few other explanatory variables i.e. not just between a pair of two explanatory variables.

```
set.seed(1)

n <- 5
m <- matrix(rnorm(n * 2), nrow = n, ncol = 2) %*% matrix(c(2, 0, -2, 3), nrow = 2, ncol = 2)
x1 <- m[,1]
x2 <- m[,2]

x1
x2
cor(x1, x2) # Not very correlated
```

```
[1] -1.2529076  0.3672866 -1.6712572  3.1905616  0.6590155
[1] -1.208498  1.095001  3.886231 -1.463218 -1.575181
[1] -0.5773345
```

```
# Construct y
y <- 1 + 2 * x1 + 3 * x2 + rnorm(n, mean = 0, sd = 0.1)

lm(y ~ x1 + x2)
```

```
Call:
lm(formula = y ~ x1 + x2)

Coefficients:
(Intercept)           x1           x2
      1.033        1.917        2.950
```

```
set.seed(2)

x3 <- x1 + 0.8 * x2 + rnorm(n, mean = 0, sd =1e-3)

x3
cor(x1, x3) # Not very correlated
cor(x2, x3)
cor(x3, x1 + 0.8 * x2) # Multicollinear
```

```
[1] -2.2206026  1.2434719  1.4393157  2.0188572 -0.6012093
[1] 0.4745505
[1] 0.4447389
[1] 0.9999998
```

```
summary(lm(y ~ x1 + x2 + x3))
```

```

Call:
lm(formula = y ~ x1 + x2 + x3)

Residuals:
         1          2          3          4          5
-0.0116829  0.1002917 -0.0474113 -0.0409079 -0.0002896

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   1.04151    0.05671  18.367   0.0346 *
x1          -81.61755  141.49937  -0.577   0.6669
x2          -63.91393  113.26165  -0.564   0.6729
x3           83.53995  141.50896   0.590   0.6605
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1188 on 1 degrees of freedom
Multiple R-squared:  0.9999,    Adjusted R-squared:  0.9996
F-statistic:  3063 on 3 and 1 DF,  p-value: 0.01328
```

- How to detect general multicollinearity?
    1. For each explanatory variable x_i, we fit a multiple regression model with x_i as the response variable to see how much it can be explained by the other explanatory variables $x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_d$.
    2. If the linear regression model R^2 is high, then the model has found a linear combination of the other explanatory variables that together is highly (in fact, highest) correlated with x_i.
    3. Compute the **variance inflation factor VIF**

$$\text{VIF} = \frac{1}{1 - R^2} \in [1, \infty)$$

    4. In practice, if $\text{VIF} > 5$ i.e. $R^2 > 0.8$ or correlation $|R| > 0.89$, then there is strong evidence of multicollinearity. In this case, we should consider omitting the explanatory variable x_i from the modeling.

```
(r2 <- summary(lm(x3 ~ x1 + x2))$r.squared)
(vif <- 1 / (1 - r2))
```

```
[1] 0.9999999
[1] 17419081
```

- Note the VIF procedure tells us if one explanatory variable is redundant when all the other explanatory variables already exist in the model. If two explanatory variables are highly correlated with each other, then they would both have high VIF values. But it doesn't mean you should simply omit both variables from modeling. Perhaps you need to keep one of them.
- Although multicollinearity often affects the estimation of model parameters (and hence model explainability and identifiability), it usually DOES NOT affect the goodness-of-fit of the model (as measured by MSE or R^2) or prediction.

```
set.seed(1)

n <- 100
x1 <- rnorm(n)
y <- 1 + 2 * x1 + rnorm(n)

m1 <- lm(y ~ x1)
summary(m1)
```

```

Call:
lm(formula = y ~ x1)

Residuals:
    Min      1Q  Median      3Q     Max 
-1.8768 -0.6138 -0.1395  0.5394  2.3462 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  0.96231    0.09699   9.922   <2e-16 ***
x1           1.99894    0.10773  18.556   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9628 on 98 degrees of freedom
Multiple R-squared:  0.7784,    Adjusted R-squared:  0.7762 
F-statistic: 344.3 on 1 and 98 DF,  p-value: < 2.2e-16
```

```
predict(m1, newdata = data.frame(x1 = 10))
```

```
      1 
20.9517 
```

```
set.seed(2)

x2 <- 2 * x1 + rnorm(n, mean = 0, sd=1e-4)

m2 <- lm(y ~ x1 + x2)
summary(m2)
```

```

Call:
lm(formula = y ~ x1 + x2)

Residuals:
    Min      1Q  Median      3Q     Max
-1.8836 -0.6172 -0.1165  0.5317  2.3326

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)    0.96216    0.09747   9.871 2.54e-16 ***
x1           334.18210 1696.40039   0.197    0.844
x2          -166.09323  848.20862  -0.196    0.845
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9675 on 97 degrees of freedom
Multiple R-squared:  0.7785,    Adjusted R-squared:  0.774
F-statistic: 170.5 on 2 and 97 DF,  p-value: < 2.2e-16
```

```
predict(m2, newdata = data.frame(x1 = 10, x2 = 20))
```

```
       1
20.91857
```

3.6 Polynomial Regression

- If the relationship between the observed response y and an explanatory variable x looks like a higher-degree polynomial instead of a straight line, then we could do **feature engineering** to construct additional explanatory variables for the higher-order terms e.g.

  $$x_1 = x,\ x_2 = x^2,\ x_3 = x^3,\ \ldots,\ x_p = x^p$$

  The resulting linear regression model

  $$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p + \epsilon$$

  is essentially fitting a pth degree polynomial curve to the original observed data x and y i.e.

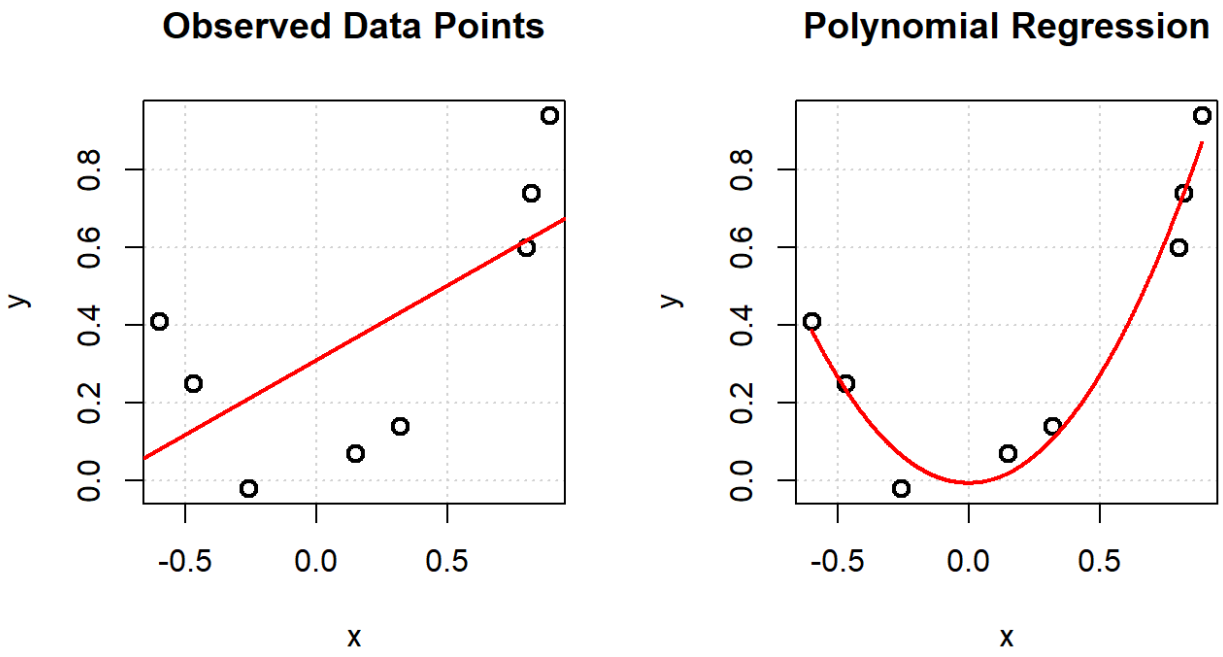
  $$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_p x^p + \epsilon$$

- Such **polynomial regression** is still considered a “linear” regression model in the sense that it involves a linear combination of the explanatory variables. It just happens that these explanatory variables $x_1,\ x_2,\ \ldots$ are related to each other.

```
x <- c(-0.47, -0.26, 0.15, 0.82, -0.6, 0.80, 0.89, 0.32)
y <- c(0.25, -0.02, 0.07, 0.74, 0.41, 0.60, 0.94, 0.14)

plot(x, y, type = 'p')
abline(lm(y ~ x), col='red')

plot(x, y, type = 'p')
model <- lm(y ~ x + I(x ^ 2))
xticks <- seq(from = min(x), to = max(x), length.out = 1000)
lines(xticks, predict(model, newdata = data.frame(x = xticks)), col='red')
```




```
summary(model)
```

```
Call:
lm(formula = y ~ x + I(x^2))

Residuals:
       1         2         3         4         5         6         7         8
 0.015154 -0.087275  0.050159  0.001398  0.022780 -0.102841  0.069301  0.031323

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.005722   0.045619  -0.125 0.905069
x            0.005357   0.063302   0.085 0.935845
I(x^2)       1.100433   0.125286   8.783 0.000317 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.07382 on 5 degrees of freedom
Multiple R-squared:  0.9669,    Adjusted R-squared:  0.9536
F-statistic: 72.94 on 2 and 5 DF,  p-value: 0.0001999
```

- In R, we can use either `I(x ^ p)` or `poly(x, p, raw = TRUE)` syntax to construct polynomial variables. Note `x ^ p` without `I()` does not work because it is for variable interaction later. For `poly()` function, we need `raw = TRUE` because otherwise it would construct the "orthogonal" version which is a complicated topic in experimental design (https://en.wikipedia.org/wiki/Design_of_experiments).

```
poly(x, 2, raw = TRUE)
```

```
         1      2
[1,] -0.47 0.2209
[2,] -0.26 0.0676
[3,]  0.15 0.0225
[4,]  0.82 0.6724
[5,] -0.60 0.3600
[6,]  0.80 0.6400
[7,]  0.89 0.7921
[8,]  0.32 0.1024
attr(,"degree")
[1] 1 2
attr(,"class")
[1] "poly"   "matrix"
```

```
summary(lm(y ~ poly(x, 2, raw = TRUE))) # Same as lm(y ~ x + I(x ^ 2))
```

```
Call:
lm(formula = y ~ poly(x, 2, raw = TRUE))

Residuals:
        1         2         3         4         5         6         7         8
 0.015154 -0.087275  0.050159  0.001398  0.022780 -0.102841  0.069301  0.031323

Coefficients:
                         Estimate Std. Error t value Pr(>|t|)
(Intercept)             -0.005722   0.045619  -0.125 0.905069
poly(x, 2, raw = TRUE)1  0.005357   0.063302   0.085 0.935845
poly(x, 2, raw = TRUE)2  1.100433   0.125286   8.783 0.000317 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.07382 on 5 degrees of freedom
Multiple R-squared:  0.9669,    Adjusted R-squared:  0.9536
F-statistic: 72.94 on 2 and 5 DF,  p-value: 0.0001999
```

- In the example above, the observed data were simulated from a pure quadratic function $y = x^2 + \epsilon$ with noise $\epsilon \sim \mathcal{N}(0, 0.01)$. The estimated intercept $\beta_0^* \approx 0$ and linear term $\beta_1^* \approx 0$ are both close to 0. Nonetheless, we should still fit the full quadratic model (with intercept and linear terms)

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$$

  instead of the pure quadratic model

$$y = \beta_2 x^2 + \epsilon$$

```
summary(lm(y ~ 0 + I(x ^ 2))) # Same as lm(y ~ I(x ^ 2) - 1)
```

```
Call:
lm(formula = y ~ 0 + I(x^2))

Residuals:
     Min       1Q   Median       3Q      Max
-0.10100 -0.02088  0.01187  0.03222  0.07240

Coefficients:
       Estimate Std. Error t value Pr(>|t|)
I(x^2)  1.09532    0.04826    22.7 8.17e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.06261 on 7 degrees of freedom
Multiple R-squared:  0.9866,    Adjusted R-squared:  0.9847
F-statistic: 515.1 on 1 and 7 DF,  p-value: 8.166e-08
```

- Why? If we apply an affine transformation $x \mapsto x + 1$ to transform the explanatory variable, then the relationship between y and x becomes

$$y = \beta_2(x+1)^2 + \epsilon = \beta_2 + 2\beta_2 x + \beta_2 x^2 + \epsilon$$

The lower-degree terms (i.e. intercept and linear terms) re-appear. Therefore, in order to preserve the invariance property of linear regression model with respect to affine transformation, it is best practice to keep all lower-degree terms $x^0 = 1,\ x,\ x^2, \ldots,\ x^{p-1}$ up to the highest degree x^p of our polynomial regression. (This continues to hold when we consider higher-order interaction terms later.)

3.7 Categorical Variable

- In order to use a categorical explanatory variable in machine learning modeling (including linear regression), it needs to be converted to numerical values first. Some common techniques are:
  1. **Label encoding**: Convert to an integer index $x \in \{1, \ldots, K\}$ or $x \in \{0, \ldots, K-1\}$ to represent one of the K categorical levels / classes.
  2. **One-hot encoding** (https://en.wikipedia.org/wiki/One-hot) / **dummy variable**: Convert to a K-dimensional binary vector $\vec{x} = (0, \ldots, 0, 1, 0, \ldots, 0) \in \{0, 1\}^K$ with the position of 1 to indicate the categorical level.
  3. Dense **embedding vector** representation $\vec{x} \in \mathbb{R}^d$ (mostly used in neural net)
- In R, the `factor(..., levels = ..., labels = ...)` function performs label encoding. (In Python, use the `sklearn.preprocessing.OrdinalEncoder()` function.)

```
blood <- c('A', 'O', 'AB', 'B', 'A', 'O', 'O', 'B', 'AB')

str(blood)
```

```
 chr [1:9] "A" "O" "AB" "B" "A" "O" "O" "B" "AB"
```

```
blood.f <- factor(blood)

blood.f
str(blood.f)
```

```
[1] A  O  AB B  A  O  O  B  AB
Levels: A AB B O
 Factor w/ 4 levels "A","AB","B","O": 1 4 2 3 1 4 4 3 2
```

```
levels(blood.f)
nlevels(blood.f)
as.character(blood.f)
as.numeric(blood.f)
```

```
[1] "A"  "AB" "B"  "O"
[1] 4
[1] "A"  "O"  "AB" "B"  "A"  "O"  "O"  "B"  "AB"
[1] 1 4 2 3 1 4 4 3 2
```

- In R, the factor levels are `sort(unique(x))` by default so the alphabetically smallest categorical level becomes the first **baseline** level. This plays a role in linear regression modeling later. To modify the baseline level, use the `relevel()` function.

```
blood.f <- relevel(blood.f, 'AB') # AB = 1, A = 2, B = 3, O = 4

blood.f
```

```
[1] A  O  AB B  A  O  O  B  AB
Levels: AB A B O
```

- To specify a custom ordering of the categorical levels, use `factor(..., levels = ...)`.

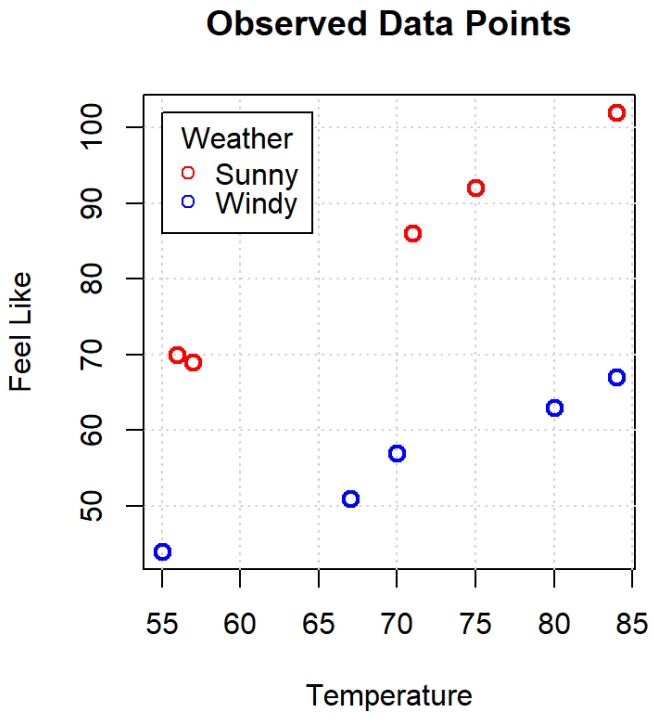
```
blood.f <- factor(blood, levels = c('O', 'AB', 'A', 'B')) # O = 1, AB = 2, BA = 3, B = 4

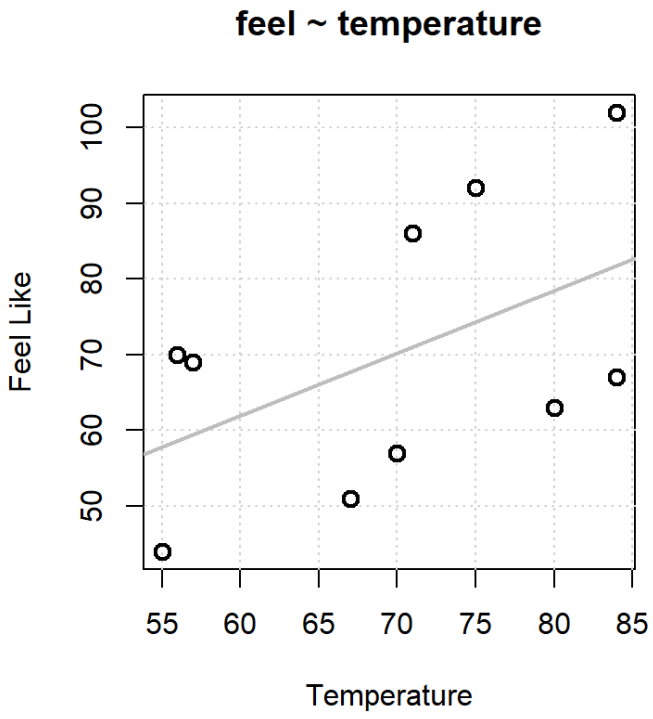
blood.f
```

```
[1] A  O  AB B  A  O  O  B  AB
Levels: O AB A B
```

- *Example:* We collect the actual temperature, feel-like temperature, and weather condition (either "Sunny" or "Windy") data on 10 days.

```
data <- data.frame(temperature = c(57, 75, 71, 56, 84, 84, 55, 80, 67, 70),
                   weather = rep(c('Sunny', 'Windy'), each = 5),
                   feel = c(69, 92, 86, 70, 102, 67, 44, 63, 51, 57))
```





- If we fit a linear regression model to all the observed data (without considering `weather` explicitly), then the model equation is

$$(\text{feel}) = 12.39 + 0.83\,(\text{temperature})$$

```
summary(lm(feel ~ temperature, data = data))
```

```
Call:
lm(formula = feel ~ temperature, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-16.706 -14.505  -1.816  14.088  20.259

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  12.3914    35.6138   0.348    0.737
temperature   0.8256     0.5038   1.639    0.140

Residual standard error: 16.84 on 8 degrees of freedom
Multiple R-squared:  0.2513,    Adjusted R-squared:  0.1578
F-statistic: 2.686 on 1 and 8 DF,  p-value: 0.1399
```

- One way to include a categorical explanatory variable in a linear regression model is as an additive term e.g. `+ weather`. In the case when there are only 2 categorical levels, this represents one (numerical) binary indicator variable

$$x_2 = \mathbf{1}_{\{\text{Windy}\}} = \begin{cases} 1 & \text{if Windy} \\ 0 & \text{if Sunny} \end{cases}$$

  This is denoted by `weatherWindy` in R's model `summary()` output.

```
summary(lm(feel ~ temperature + weather, data = data))
```

```
Call:
lm(formula = feel ~ temperature + weather, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.1979 -1.9510 -0.6991  1.7999  3.8029

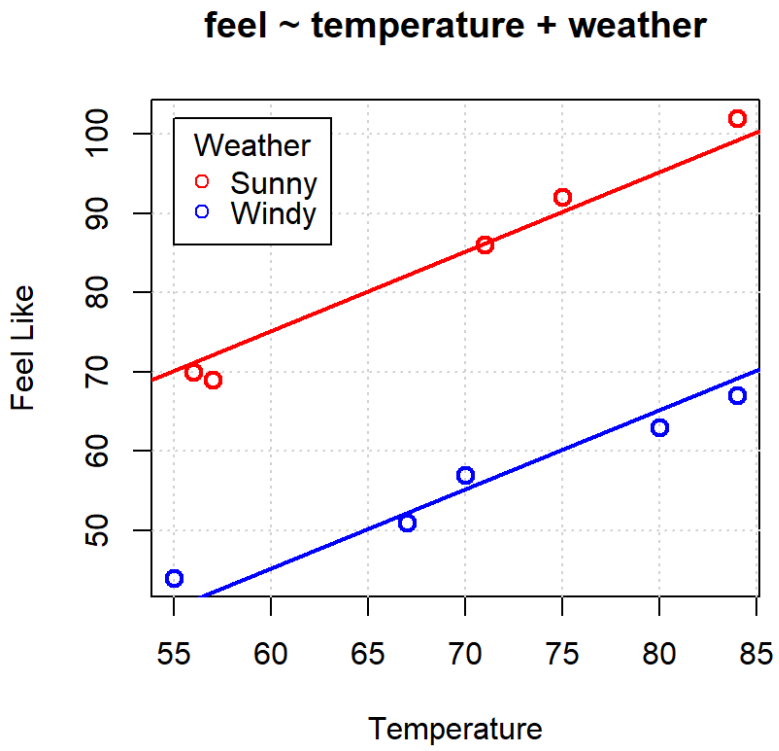
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)   15.18753    5.74546   2.643   0.0333 *
temperature    1.00018    0.08186  12.218 5.63e-06 ***
weatherWindy -30.00047    1.73028 -17.339 5.22e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.715 on 7 degrees of freedom
Multiple R-squared:  0.983, Adjusted R-squared:  0.9781
F-statistic: 201.9 on 2 and 7 DF,  p-value: 6.453e-07
```

- The model equation is

$$\begin{aligned}(\text{feel}) &= 15.19 + 1.00\,(\text{temperature}) - 30.00\,\mathbf{1}_{\{\text{Windy}\}} \\ &= \begin{cases} 15.19 + 1.00\,(\text{temperature}) & \text{if Sunny} \\ -14.81 + 1.00\,(\text{temperature}) & \text{if Windy} \end{cases}\end{aligned}$$

  It essentially fits two separate lines with the same slope β_1 but different intercepts β_0 vs $\beta_0 + \beta_2$ to the observed data.



- Here β_0 is the intercept for the "Sunny" group while β_2 is the increment (or decrement) of the intercept of the "Windy" group relative to the "Sunny" group. In this sense, the "Sunny" group is the baseline. However, there is nothing intrinsically special about the baseline group. "Sunny" is selected as the baseline merely because it is the first level in `factor(weather)`.
- If we `relevel()` to "Windy" first, then the model equation becomes

$$\begin{aligned}(\text{feel}) &= -14.81 + 1.00\,(\text{temperature}) + 30.00\,\mathbf{1}_{\{\text{Sunny}\}} \\ &= \begin{cases} -14.81 + 1.00\,(\text{temperature}) & \text{if Windy} \\ 15.19 + 1.00\,(\text{temperature}) & \text{if Sunny} \end{cases}\end{aligned}$$

  It is equivalent to the previous model equation.

```
(data$weather <- relevel(factor(data$weather), 'Windy'))
```

```
 [1] Sunny Sunny Sunny Sunny Sunny Windy Windy Windy Windy Windy
Levels: Windy Sunny
```

```
summary(lm(feel ~ temperature + weather, data = data))
```

```
Call:
lm(formula = feel ~ temperature + weather, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.1979 -1.9510 -0.6991  1.7999  3.8029

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -14.81295    5.95366  -2.488   0.0417 *
temperature    1.00018    0.08186  12.218 5.63e-06 ***
weatherSunny  30.00047    1.73028  17.339 5.22e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.715 on 7 degrees of freedom
Multiple R-squared:  0.983, Adjusted R-squared:  0.9781
F-statistic: 201.9 on 2 and 7 DF,  p-value: 6.453e-07
```

- Another way to fit the same linear regression model is to omit the default intercept term β_0 by either specifying `0 +` or `- 1` in R's formula. The model equation is

$$(\text{feel}) = 0 + 1.00\,(\text{temperature}) - 14.81\,\mathbf{1}_{\{\text{Windy}\}} + 15.19\,\mathbf{1}_{\{\text{Sunny}\}}$$

  which has separate intercepts β_2 and β_3 for each `weather` group explicitly without one particular group as the baseline.

```
summary(lm(feel ~ 0 + temperature + weather, data = data))
```

```
Call:
lm(formula = feel ~ 0 + temperature + weather, data = data)

Residuals:
    Min      1Q  Median      3Q     Max
-3.1979 -1.9510 -0.6991  1.7999  3.8029

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
temperature    1.00018    0.08186  12.218 5.63e-06 ***
weatherWindy -14.81295    5.95366  -2.488   0.0417 *
weatherSunny  15.18753    5.74546   2.643   0.0333 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.715 on 7 degrees of freedom
Multiple R-squared:  0.999, Adjusted R-squared:  0.9986
F-statistic:  2357 on 3 and 7 DF,  p-value: 7.082e-11
```

- In this case, the design matrix $X \in \mathbb{R}^{10\times 3}$ does not have a first column of 1's anymore. Instead, it has 2 columns of binary indicator variables

$$x_2 = \mathbf{1}_{\{\text{Windy}\}} = \begin{cases} 1 & \text{if Windy} \\ 0 & \text{if Sunny} \end{cases} \qquad x_3 = \mathbf{1}_{\{\text{Sunny}\}} = \begin{cases} 1 & \text{if Sunny} \\ 0 & \text{if Windy} \end{cases}$$

Together, they form the one-hot encoding / dummy variables of the `weather` categorical variable. (In Python, use `sklearn.preprocessing.OneHotEncoder()` or `pandas.get_dummies()` functions.)

```
(X <- model.matrix(feel ~ 0 + temperature + weather, data = data))
```

```
   temperature weatherWindy weatherSunny
1           57            0            1
2           75            0            1
3           71            0            1
4           56            0            1
5           84            0            1
6           84            1            0
7           55            1            0
8           80            1            0
9           67            1            0
10          70            1            0
attr(,"assign")
[1] 1 2 2
attr(,"contrasts")
attr(,"contrasts")$weather
[1] "contr.treatment"
```

- Why not have both the (constant) intercept variable $x_0 = 1$ along with one-hot encoding of all levels $x_2 = \mathbf{1}_{\{\text{Windy}\}}$ and $x_3 = \mathbf{1}_{\{\text{Sunny}\}}$ of a categorical variable in the model equation?

  $$(\text{feel}) = \beta_0 + \beta_1 \, (\text{temperature}) + \beta_2 \, \mathbf{1}_{\{\text{Windy}\}} + \beta_3 \, \mathbf{1}_{\{\text{Sunny}\}}$$

  It would have multicollinearity because

  $$x_0 = 1 = \mathbf{1}_{\{\text{Windy}\}} + \mathbf{1}_{\{\text{Sunny}\}} = x_2 + x_3$$

  In this case, the model would be **over-parameterized** / **unidentifiable** because the intercepts for the two groups are now $\beta_0 + \beta_2$ and $\beta_0 + \beta_3$ which have no unique optimal parameters.
- In summary, for categorical variable, we must drop either one level of the one-hot encoding or the default intercept in our design matrix X.
- What if we fit a linear regression model with only a categorical variable?

  $$(\text{feel}) = 0 + \beta_1 \, \mathbf{1}_{\{\text{Windy}\}} + \beta_2 \, \mathbf{1}_{\{\text{Sunny}\}}$$

  In statistics literature, this is called a **fixed-effect** model.

```
summary(lm(feel ~ 0 + weather, data = data))
```

```
Call:
lm(formula = feel ~ 0 + weather, data = data)

Residuals:
   Min     1Q Median     3Q    Max
-14.80 -10.65   1.40   7.80  18.20

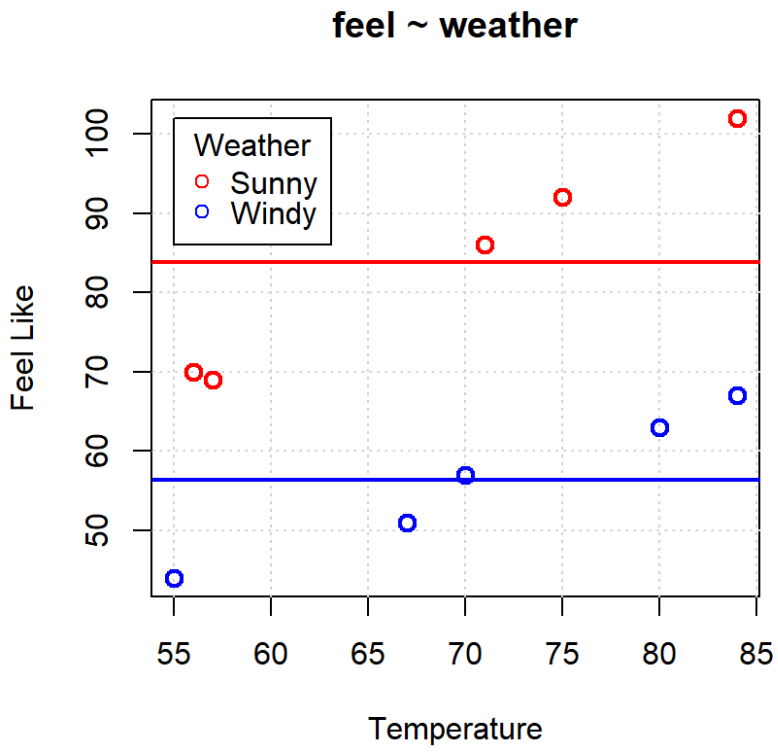
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
weatherWindy   56.400      5.367   10.51 5.85e-06 ***
weatherSunny   83.800      5.367   15.62 2.82e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 12 on 8 degrees of freedom
Multiple R-squared:  0.9779,    Adjusted R-squared:  0.9724
F-statistic: 177.1 on 2 and 8 DF,  p-value: 2.378e-07
```

- The optimal intercept parameters β_1^* and β_2^* are simply the per-group sample means.

```
aggregate(feel ~ weather, data = data, mean) # with(data, tapply(feel, weather, mean))
```

```
  weather feel
1   Windy 56.4
2   Sunny 83.8
```



3.8 Interaction

- An **interaction** term is a kind of feature engineering in which two (or more) explanatory variables are multiplied together to form a new explanatory variable. This is more interesting when one explanatory variable is quantitative / continuous while another is categorical.
- In R's formula, use the `x1:x2` notation to denote a **2nd order** / **two-way** interaction term between two explanatory variables `x1` and `x2`. Meanwhile, the **1st order** terms `x1` and `x2` are called the **main effects**.

```
summary(lm(feel ~ temperature + weather + temperature:weather, data = data))
```

```
Call:
lm(formula = feel ~ temperature + weather + temperature:weather,
    data = data)

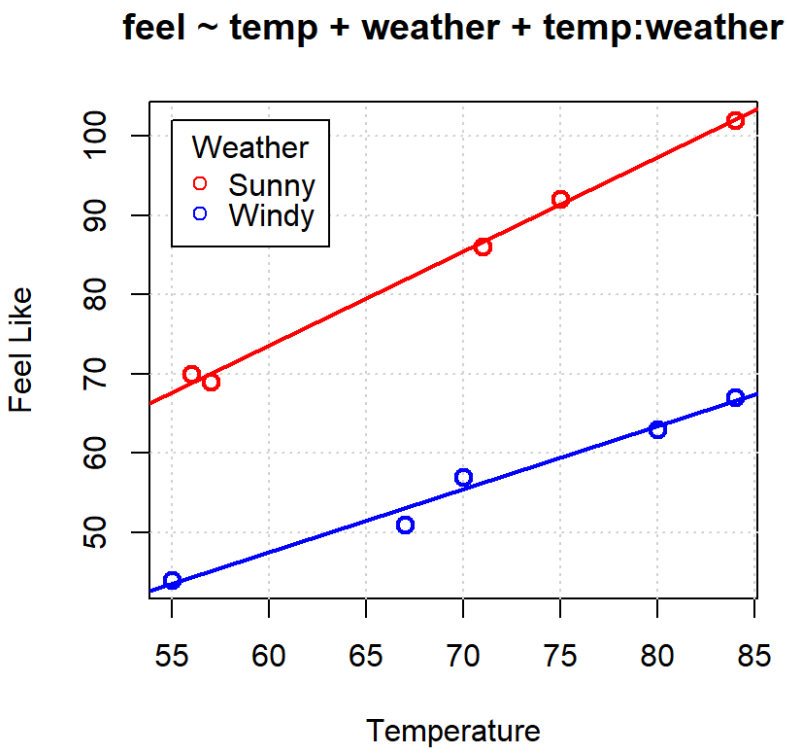
Residuals:
    Min      1Q  Median      3Q     Max
-2.0532 -0.5849  0.1807  0.5925  1.5562

Coefficients:
                          Estimate Std. Error t value Pr(>|t|)
(Intercept)                2.55440    3.80751   0.671  0.52726
temperature                1.18434    0.05483  21.598 6.43e-07 ***
weatherWindy              -2.89105    5.62791  -0.514  0.62583
temperature:weatherWindy -0.38748    0.07954  -4.871  0.00279 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.317 on 6 degrees of freedom
Multiple R-squared:  0.9966,    Adjusted R-squared:  0.9948
F-statistic: 579.7 on 3 and 6 DF,  p-value: 8.878e-08
```

- The model equation is

$$\begin{aligned}(\text{feel}) &= 2.55 + 1.18\,(\text{temperature}) - 2.89\,\mathbf{1}_{\{\text{Windy}\}} - 0.39\,(\text{temperature})\,\mathbf{1}_{\{\text{Windy}\}} \\ &= (2.55 - 2.89\,\mathbf{1}_{\{\text{Windy}\}}) + (1.18 - 0.39\,\mathbf{1}_{\{\text{Windy}\}})\,(\text{temperature}) \\ &= \begin{cases} 2.55 + 1.18\,(\text{temperature}) & \text{if Sunny} \\ -0.34 + 0.80\,(\text{temperature}) & \text{if Windy} \end{cases}\end{aligned}$$



- This linear regression model is exactly the same as fitting two separate linear regression models to each group of the observed data.

```
lm(feel ~ temperature, data = data, subset = weather == 'Sunny')
lm(feel ~ temperature, data = data, subset = weather == 'Windy')
```

```

Call:
lm(formula = feel ~ temperature, data = data, subset = weather ==
    "Sunny")

Coefficients:
(Intercept)  temperature
      2.554        1.184


Call:
lm(formula = feel ~ temperature, data = data, subset = weather ==
    "Windy")

Coefficients:
(Intercept)  temperature
    -0.3366       0.7969
```

- Without any interaction, the `temperature` explanatory variable affects the slope while the `weather` explanatory variable affects the intercept of the model separately. With `temperature:weather` interaction term, the overall effect is no longer simply the additive effect of each part. They "interact" in this sense.
- In the design matrix $X \in \mathbb{R}^{10\times 4}$, the last column $x_3 = x_1 \times x_2$ is simply the product of the other two `temperature` and `weatherWindy` columns.

```
(X <- model.matrix(feel ~ temperature + weather + temperature:weather, data = data))
```

```
   (Intercept) temperature weatherWindy temperature:weatherWindy
1            1          57            0                        0
2            1          75            0                        0
3            1          71            0                        0
4            1          56            0                        0
5            1          84            0                        0
6            1          84            1                       84
7            1          55            1                       55
8            1          80            1                       80
9            1          67            1                       67
10           1          70            1                       70
attr(,"assign")
[1] 0 1 2 3
attr(,"contrasts")
attr(,"contrasts")$weather
[1] "contr.treatment"
```

- In R, we can use the shorthand notation `x1 * x2` for `x1 + x2 + x1:x2` .

```
lm(feel ~ temperature * weather, data = data)
```

```

Call:
lm(formula = feel ~ temperature * weather, data = data)

Coefficients:
            (Intercept)              temperature              weatherWindy
                 2.5544                   1.1843                   -2.8910
temperature:weatherWindy
                -0.3875
```

- Another way to fit the same linear regression model is to not drop one level of the categorical variable. To do so, we use some R's formula algebra to omit those terms that would cause multicollinearity. The model equation is

$$\begin{aligned}(\text{feel}) &= 2.55\,\mathbf{1}_{\{\text{Sunny}\}} - 0.34\,\mathbf{1}_{\{\text{Windy}\}} + 1.18\,(\text{temperature})\,\mathbf{1}_{\{\text{Sunny}\}} + 0.80\,(\text{temperature})\,\mathbf{1}_{\{\text{Sunny}\}} \\ &= (2.55 + 1.18\,(\text{temperature}))\;\mathbf{1}_{\{\text{Sunny}\}} + (-0.34 + 0.80\,(\text{temperature}))\;\mathbf{1}_{\{\text{Windy}\}} \\ &= \begin{cases} 2.55 + 1.18\,(\text{temperature}) & \text{if Sunny} \\ -0.34 + 0.80\,(\text{temperature}) & \text{if Windy} \end{cases}\end{aligned}$$

```
# feel ~ 1 + temperature + weather + temperature:weather
lm(feel ~ temperature * weather - 1 - temperature, data = data)
lm(feel ~ 0 + weather + temperature:weather, data = data) # No constant intercept
```

```
Call:
lm(formula = feel ~ temperature * weather - 1 - temperature,
    data = data)

Coefficients:
            weatherSunny              weatherWindy  temperature:weatherSunny
                  2.5544                   -0.3366                    1.1843
temperature:weatherWindy
                  0.7969


Call:
lm(formula = feel ~ 0 + weather + temperature:weather, data = data)

Coefficients:
            weatherSunny              weatherWindy  weatherSunny:temperature
                  2.5544                   -0.3366                    1.1843
weatherWindy:temperature
                  0.7969
```

- It is possible to have multi-way and higher-order interaction terms too. See `?formula`. For example:
  - `x1 * x2 * x3` represents `x1 + x2 + x3 + x1:x2 + x1:x3 + x2:x3 + x1:x2:x3` all lower-order interactions up to the highest-order all-way interaction term. (This is the reason why the `I()` notation is needed to distinguish this from `I(x1 * x2 * x3)`.)
  - `(x1 + x2 + x3) ^ 2` represents `x1 + x2 + x3 + x1:x2 + x1:x3 + x2:x3` up to 2nd-order interactions only.

4 Statistical Model

- A **statistical model** assumes the observed data are **realizations** of random variables from some unknown probability distribution.
- We usually assume the set of observed data $x_1, \ldots, x_n$ is an **i.i.d. sample** from some population. It means the corresponding random variables $X_1, \ldots, X_n$ are independent and identically distributed with the same probability distribution (denoted by its cumulative distribution function) F which describes the overall population.

4.1 Bernoulli $\mathrm{Bern}(p)$

- A random variable $X \sim \mathrm{Bern}(p)$ with "mean" parameter $0 < p < 1$ only takes on either 0 or 1 value (its **domain** or **support**). It describes a binary outcome e.g. 1 = success, 0 = failure.
- *Example:* Let A be an event that occurs randomly e.g. $A =$ "$\mathrm{rain\ tomorrow}$". The binary indicator variable

$$\mathbf{1}_{\{\mathrm{rain\ tomorrow}\}} = \mathbf{1}_A = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{if } A \text{ does not occur} \end{cases} \sim \mathrm{Bern}(p)$$

  is a Bernoulli random variable with parameter $p = P(A)$.

```
# Simulate Bern(p) observations
p <- 0.3
rbinom(10, size = 1, prob = p) # size=1 since Bern(p) = Binom(n=1,p)
```

```
[1] 1 0 0 1 0 1 0 0 0 0
```

- Probability mass function p_X can be written in two equivalent ways:

$$P(X = x) = p_X(x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \end{cases}$$
$$= p^x (1-p)^{1-x}$$

  The 2nd way is useful for writing the likelihood function and differentiate to compute maximum likelihood estimate MLE later.
- Mean $\mathbb{E}(X)$ and variance $\mathrm{Var}(X)$ are

$$\mathbb{E}(X) = 1 \times p + 0 \times (1-p) = p$$
$$\mathrm{Var}(X) = \mathbb{E}(X-p)^2 = p(1-p)$$

  Note $\mathrm{Var}(X) \approx 0$ when $p \approx 0$ or 1. Highest variance $\mathrm{Var}(X) = 1/4$ when $p = 0.5$ i.e. X has most uncertainty.
- *Example:* For a binary indicator random variable $\mathbf{1}_A$

$$\mathbb{E}(\mathbf{1}_A) = p = P(A)$$

- CDF F_X is two-step with jumps of $1-p$ at $x = 0$ and p at $x = 1$
- In practice, it is often used to represent the output of binary classification model such as logistic regression (https://en.wikipedia.org/wiki/Logistic_regression).

4.2 Binomial $\mathrm{Binom}(n, p)$

- A random variable $X \sim \mathrm{Binom}(n, p)$ with fixed (not a parameter) n number of trials and parameter $0 < p < 1$ takes on value within $\{0, \ldots, n\}$. It describes the number of successes in n independent Bernoulli trials e.g. number of heads in n coin tosses.

```
# Simulate Binom(n,p) observations
n <- 10
p <- 0.6
rbinom(20, size = n, prob = p)
```

```
 [1] 3 6 5 5 3 8 7 9 7 7 8 4 6 6 5 4 4 6 5 5
```

- Probability mass function p_X is

$$P(X = x) = p_X(x) = \binom{n}{x} p^x (1-p)^{n-x} = P(x \text{ successes in } n \text{ trials})$$

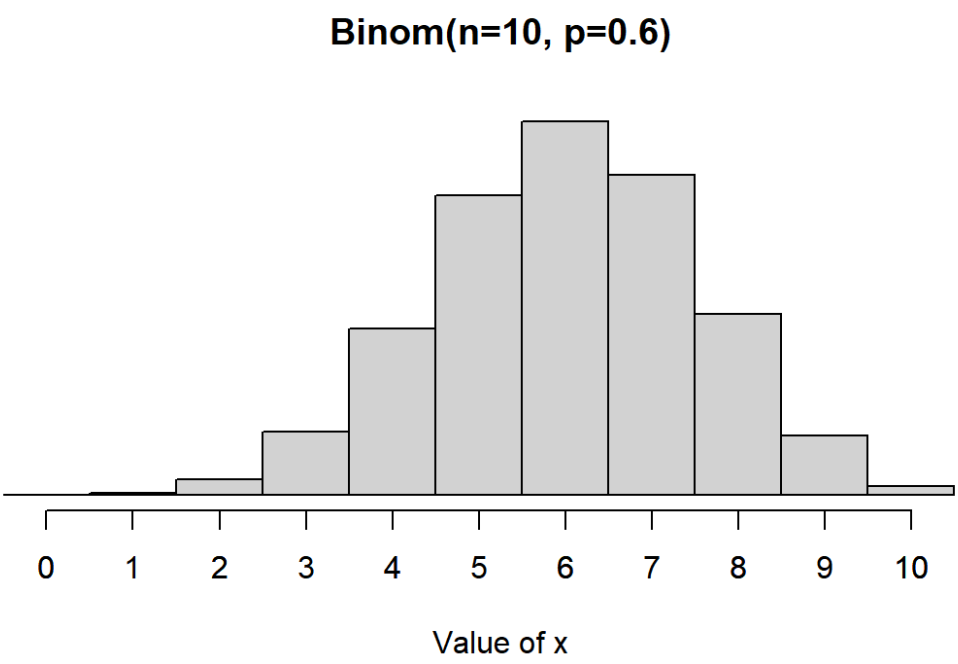
- "Binomial is a sum of Bernoulli" i.e. if $X_1, \ldots, X_n \sim \mathrm{Bern}(p)$ are independent, then

$$X_1 + \cdots + X_n \sim \mathrm{Binom}(n, p)$$

- By linearity of expectation and variance (for independent random variables),

$$\mathbb{E}(X) = \mathbb{E}(X_1) + \cdots + \mathbb{E}(X_n) = np$$
$$\mathrm{Var}(X) = \mathrm{Var}(X_1) + \cdots + \mathrm{Var}(X_n) = np(1-p)$$

- Probability mass function p_X is unimodal and centered near its mean $\mathbb{E}(X) = np$.



- In practice, it is often used in statistical hypothesis testing (also called A/B test in the industry) e.g. how many customers in a treatment group make purchases.

4.3 Uniform $\mathrm{Unif}(a, b)$

- A random variable $X \sim \mathrm{Unif}(a, b)$ with parameters $-\infty < a < b < \infty$ has support on the $[a, b]$ interval.

```
# Simulate Unif(-1,2) observations
a <- -1
b <- 2
runif(5, min = a, max = b)
```

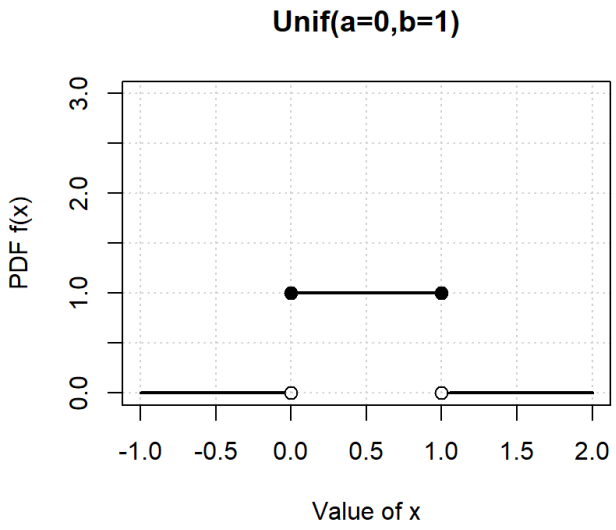
```
[1] 0.6816963 0.6983556 0.7439175 1.9898510 0.3372242
```

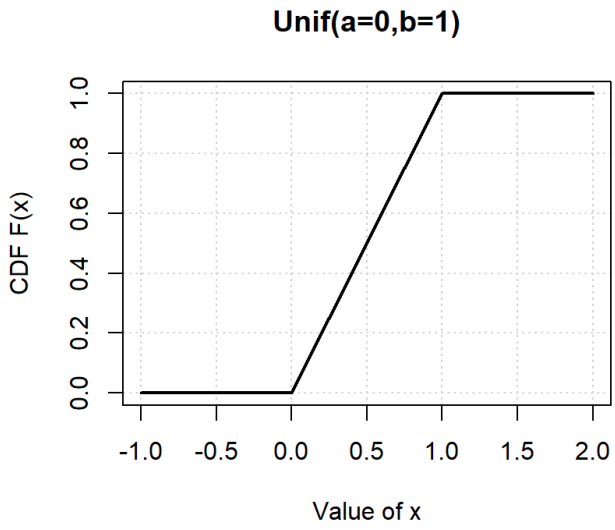
- Probability density function f_X is

$$f_X(x) = \begin{cases} 1/(b-a) & \text{if } a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

- Mean value $\mathbb{E}(X)$ is the midpoint of the $[a, b]$ interval

$$\mathbb{E}(X) = \frac{a+b}{2}$$



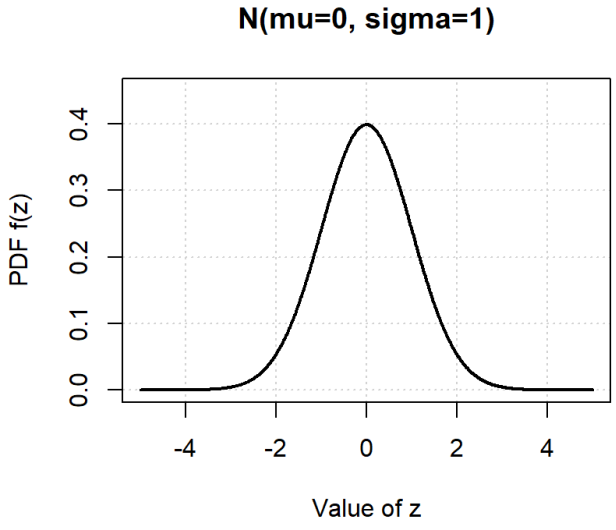


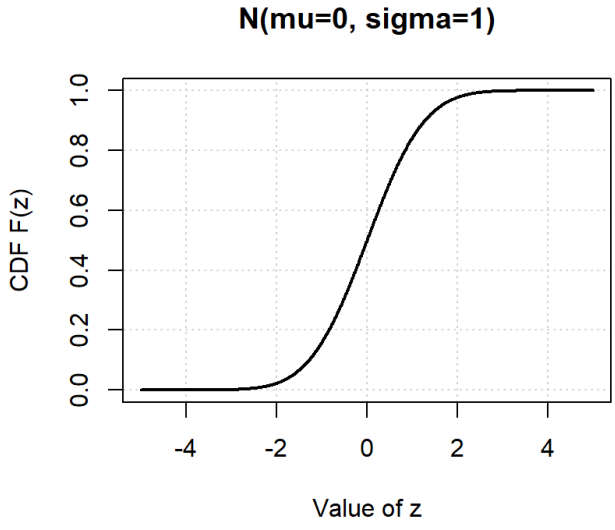
4.4 Univariate Gaussian / normal $\mathcal{N}(\mu, \sigma^2)$

- A **standard Gaussian** or **normal** random variable $Z \sim \mathcal{N}(0, 1)$ has probability density function

$$f_Z(z) = \phi(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}$$

with support on the entire real number line $\mathbb{R} = (-\infty, \infty)$. Its cumulative distribution function $F_Z(x) = \Phi(z)$ is an integral but does not have a simple closed-form formula.





- Probability density function $\phi(z)$ is a unimodal **bell-shaped** curve. Since it is symmetric around $z = 0$, the mean / median / mode are all equal.

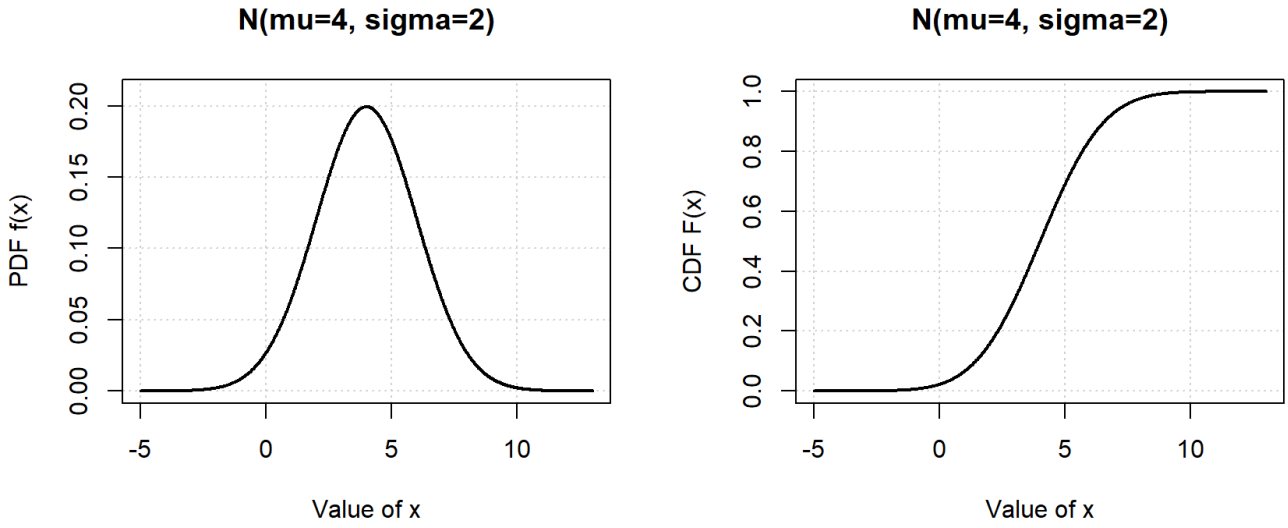
$$\mathbb{E}(Z) = 0 \qquad \mathrm{Var}(Z) = 1 = \mathrm{SD}(Z)$$

- In practice, the cumulative distribution function $\Phi(z)$ is sometimes used as a mathematical function to "squeeze" the real line $\mathbb{R}$ monotonically onto the unit interval $(0, 1)$.
- Let $-\infty < \mu < \infty$ be the **location** parameter and $\sigma > 0$ be the **scale** parameter. By performing an **affine transformation** to first scale and then translate Z, we obtain a general normal random variable

$$X = \mu + \sigma Z \sim \mathcal{N}(\mu, \sigma^2)$$

with probability density function

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$




```
# Simulate N(4,4) observations
mu <- 4
sigma <- 2
rnorm(5, mean = mu, sd = sigma)
```

```
[1] 2.015505 2.993980 3.951921 3.942039 4.895343
```

- By linearity of expectation and variance,

$$\begin{aligned}\mathbb{E}(X) &= \mu + \sigma\mathbb{E}(Z) = \mu \\ \mathrm{Var}(X) &= \sigma^2 \mathrm{Var}(Z) = \sigma^2 \\ \mathrm{SD}(X) &= \sigma\end{aligned}$$

- Conversely, if $X \sim \mathcal{N}(\mu, \sigma^2)$, then we can reverse the transformation to obtain the standard normal

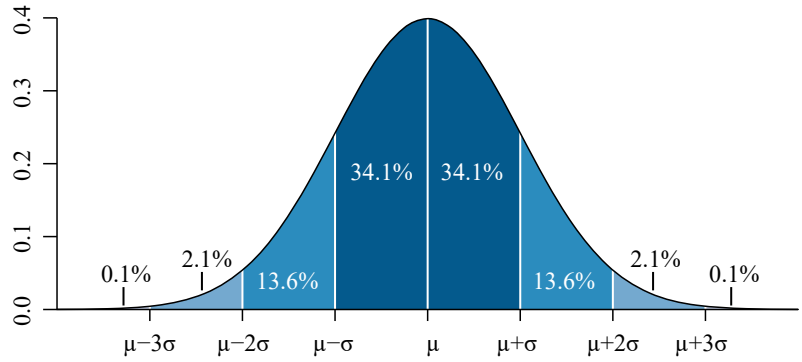
$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1)$$

- In general, for any random variable X, this kind of transformation

$$Y = \frac{X - \mathbb{E}(X)}{\mathrm{SD}(X)}$$

  is called **standardization** or **normalization**. The resulting random variable Y always has $\mathbb{E}(Y) = 0$ and $\mathrm{Var}(Y) = \mathrm{SD}(Y) = 1$.
- (Continuing with normal distribution.) Probability density function f_X is centered at $x = \mu$ with **inflection points** at $x = \mu \pm \sigma$.

- **68% 95% 99.7% rule**: The probability of an observation (or area under the curve) falling within ± 1 (or ± 2 or ± 3) σ standard deviation(s) from the mean μ is approximately 68.3%, 95.5%, and 99.7% respectively.



(Source (https://commons.wikimedia.org/wiki/File:Standard_deviation_diagram_micro.svg))

- Probability density function f_X has a **thin tail** since it decays very fast like e^{-x^2}. In practice, this means the chance of observing data that is "more than a few standard deviations away from the mean" is unlikely.
- A mathematical way to measure the tail heaviness of a probability distribution is **kurtosis**

$$\mathrm{Kurt}(X) = \frac{\mathbb{E}\left((X-\mu)^4\right)}{\left(\mathbb{E}(X-\mu)^2\right)^2} > 0$$

Larger kurtosis value means heavier tail.

- A normal $\mathcal{N}(\mu, \sigma^2)$ distribution always has kurtosis $\mathrm{Kurt}(X) = 3$.
- Linear combination: If $X_1, \ldots, X_n$ are independent $N(\mu_i, \sigma_i^2)$ random variables and $a_1, \ldots, a_n$ are any constants, then

$$a_1 X_1 + \cdots + a_n X_n \sim \mathcal{N}(a_1\mu_1 + \cdots + a_n\mu_n,\, a_1^2\sigma_1^2 + \cdots + a_n^2\sigma_n^2)$$

Conversely, if $X \perp Y$ are independent and their sum $X + Y$ is normally distributed, then X and Y are also normally distributed (individually).

- The most important role of the normal distribution is due to the central limit theorem (https://en.wikipedia.org/wiki/Central_limit_theorem) CLT.

4.5 Inference

- A parametric statistical model F_θ is indexed by the parameter θ. The set Θ of all possible parameters of interest (or in consideration) is called the **parameter space**.
- **Statistical inference** is the task of using the observed data to:
  - Compute **point estimate** $\hat{\theta}$ of the unknown parameter θ (or some related quantity) within the parameter space Θ
  - Construct **confidence interval** to describe the uncertainty of estimation
  - Perform **hypothesis testing** to assess the plausibility of the unknown parameter θ being equal to some hypothesized value $\theta_0 \in \Theta$
- *Example:* Let $x_1, \ldots, x_n$ be an i.i.d. sample from $\mathcal{N}(\mu, \sigma^2)$ distribution with unknown mean μ and variance σ^2 parameters. It might make sense to estimate μ by the **sample mean**

$$\hat{\mu} = \bar{x} \equiv \frac{1}{n}\sum_{i=1}^{n} x_i$$

and estimate σ^2 by the **(unadjusted) sample variance**

$$\hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

```
n <- 100 # Number of observed data
x <- rnorm(n, mean = 7, sd = 2) # N(7,4)

(x.bar <- mean(x))

mean((x - x.bar)^2)
```

```
[1] 6.734491
[1] 3.258627
```

- For given values of the observed data $x_1, \ldots, x_n$, their sample mean $\bar{x}$ is an **estimate** of the unknown true mean parameter μ. But if we view these observed data as a particular realized sample of the corresponding random variables $X_1, \ldots, X_n$, then

$$\hat{\theta} = \bar{X} \equiv \frac{1}{n}\sum_{i=1}^{n} X_i$$

is also a random variable. This is called an **estimator**. It describes a recipe to estimate the unknown parameter θ.

4.6 Biasedness

- One desirable property of an estimator $\hat{\theta}$ is to be **unbiased** i.e.

$$\mathbb{E}_\theta(\hat{\theta}) = \theta \quad \text{for all } \theta \in \Theta$$

Here $\mathbb{E}_\theta$ means the expected value when the true parameter is θ. Equality needs to hold no matter what the true parameter $\theta \in \Theta$ is.

- *Example:* The i.i.d. sample mean is always an unbiased estimator of the unknown true mean parameter. Why? By linearity of expectation, if $X_1, \ldots, X_n$ is an i.i.d. sample from probability distribution F with mean $\mathbb{E}(X) = \mu$, then

$$\mathbb{E}_\mu(\bar{X}) = \mathbb{E}_\mu\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \mu$$

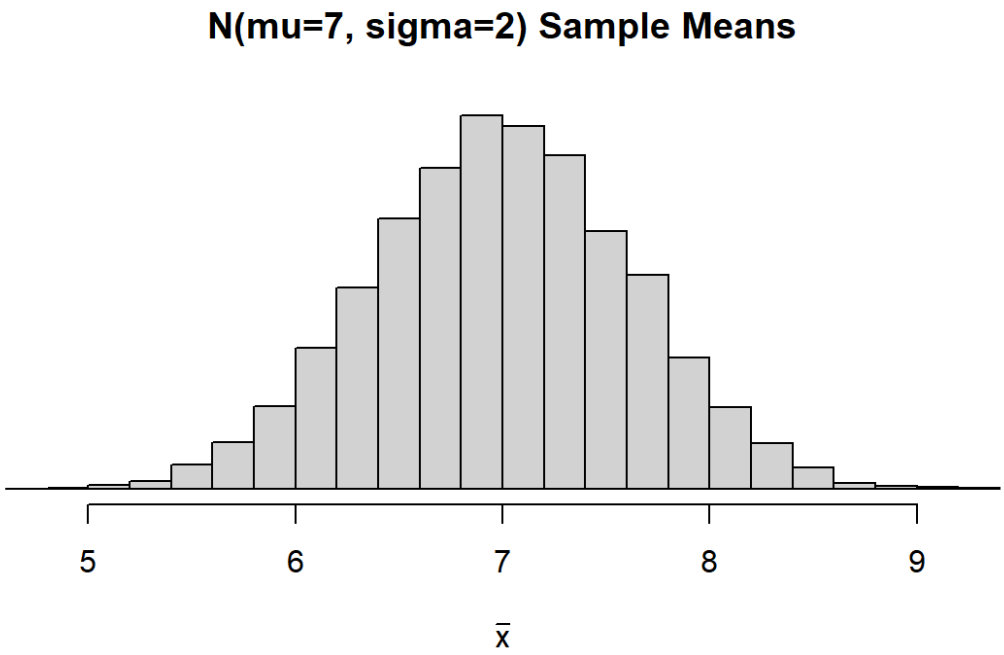
no matter what μ is.

```
set.seed(1)

num.of.estimate <- 10000
sample.means <- numeric(num.of.estimate)

for (i in seq_len(num.of.estimate)) {
  n <- 10
  x <- rnorm(n, mean = 7, sd = 2) # N(7,4) with mu = 7
  sample.means[i] <- mean(x) # Sample mean estimator
}

hist(sample.means,
     breaks=25,
     xlab=expression(bar(x)),
     yaxt='n', ylab='',
     main='N(mu=7, sigma=2) Sample Means')
```




```
mean(sample.means) # E(sample mean estimator) = mu
```

```
[1] 6.995512
```

- *Broken Clock Example:* Suppose we propose to use a (constant) estimator $\hat{p} = 0.4$ for the $\text{Bern}(p)$ distribution i.e. no matter what the unknown true parameter $0 < p < 1$ is or what the observed sample data $x_1, \ldots, x_n$ are, we just say "our estimate of p is always 0.4". This constant estimator is correct only when $p = 0.4$. Otherwise, it would be either an under-estimate or over-estimate in other cases when $p \neq 0.4$. So it is not unbiased for all values of p in the parameter space $\Theta = (0, 1)$.
- In general, the **bias** of an estimator $\hat{\theta}$ is

$$\text{Bias}(\hat{\theta}) = \mathbb{E}_\theta(\hat{\theta}) - \theta$$

- *Example:* The (unadjusted) sample variance is always a biased estimator for the unknown true variance σ^2.

$$\mathbb{E}(\hat{\sigma}^2) = \mathbb{E}\left(\frac{1}{n}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^2\right) = \ldots = \frac{n-1}{n}\sigma^2 < \sigma^2$$

  In other words, the (unadjusted) sample variance tends to underestimate the true variance. Intuitively, since the n summands

$$\sum_{i=1}^{n}(X_i - \bar{X}) = \left(\sum_{i-1}^{n} X_i\right) - n\bar{X} = 0$$

  i.e. they must sum to 0. The last summand is completely determined by the first $n-1$ terms. There are only $n-1$ "independent things" or **degrees of freedom**. (In linear algebra, it is a d-dimensional vector in a $(d-1)$-dimensional subspace.) So it makes sense to divide by $n-1$ instead of n.
- *Example:* The (adjusted) **sample variance**

$$\hat{\sigma}^2 = \frac{1}{n-1}\sum_{i=1}^{n}\left(x_i - \bar{x}\right)^2$$

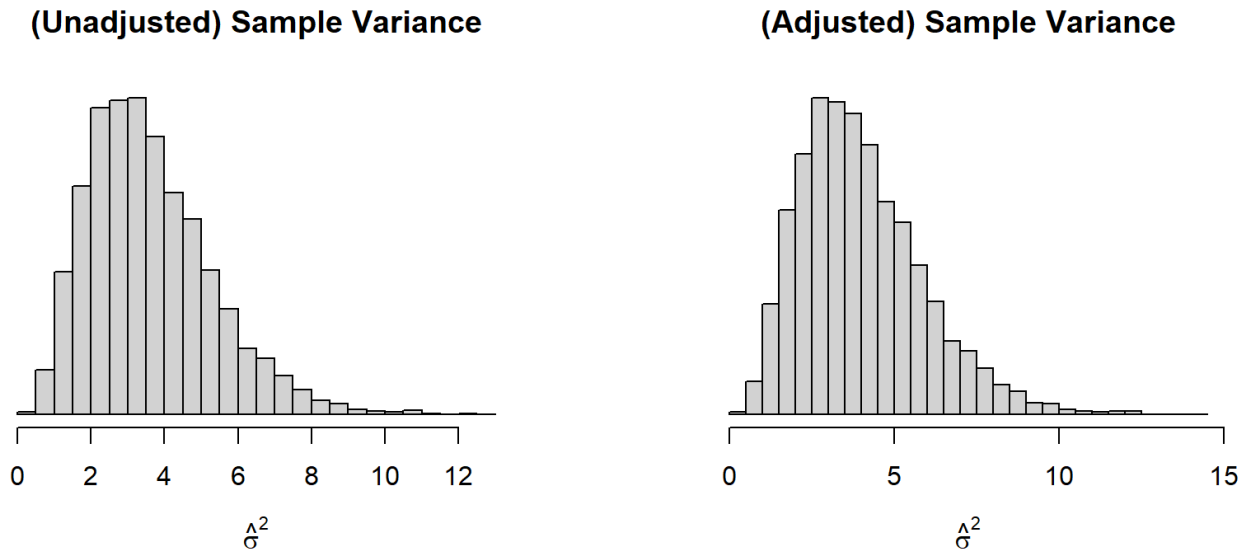
  is always an unbiased biased estimator for the unknown true variance σ^2. In other words,

$$\mathbb{E}(\hat{\sigma}^2) = \mathbb{E}\left(\frac{1}{n-1}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^2\right) = \sigma^2$$

```
num.of.estimate <- 10000
unadjusted.sample.var <- numeric(num.of.estimate)
sample.var <- numeric(num.of.estimate)

for (i in seq_len(num.of.estimate)) {
  n <- 10
  x <- rnorm(n, mean = 7, sd = 2) # N(7,4) with mu = 7
  unadjusted.sample.var[i] <- mean((x - mean(x))^2) # Unadjusted sample variance
  sample.var[i] <- sum((x - mean(x))^2) / (n - 1) # Adjusted sample variance
}

par(mfrow=c(1,2))
hist(unadjusted.sample.var,
     breaks=20,
     xlab=expression(hat(sigma)^2),
     yaxt='n', ylab='',
     main='(Unadjusted) Sample Variance')
hist(sample.var,
     breaks=20,
     xlab=expression(hat(sigma)^2),
     yaxt='n', ylab='',
     main='(Adjusted) Sample Variance')
```




```
mean(unadjusted.sample.var)
mean(sample.var)
```

```
[1] 3.613107
[1] 4.014564
```

- In practice, this concept of biasedness also appears in machine learning as the bias-variance tradeoff (https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff). It says that, while it is a good characteristic for an estimator to be unbiased, sometimes we can trade a bit of increase in biasedness for much reduced variance and hence better overall result.

4.7 Likelihood Principle

- Let $x_1, \ldots, x_n$ be a set of n observed i.i.d. sample data points from a parametric statistical model F_θ with unknown true parameter θ. We would like to have a principled approach to perform parameter estimation and statistical inference. This is in contrast to ad-hoc approach e.g. "if θ is the true mean, then let's estimate it using something that looks alike e.g. sample mean" without knowing why this might (or might not) be the ideal thing to do.
- The **likelihood function** is

$$L(\theta) = P_\theta(X_1 = x_1, \ldots, X_n = x_n) = P_\theta(X_1 = x_1) \times \cdots \times P_\theta(X_n = x_n) = \prod_{i=1}^{n} P_\theta(X_i = x_i)$$

  In other words, it is the probability (or likeliness) of observing the given observed data under the model parameter θ. We usually view this quantity as a function of θ with fixed data.
- The **log-likelihood function** is

$$l(\theta) = \log L(\theta) = \log \prod_{i=1}^{n} P_\theta(X_i = x_i) = \sum_{i=1}^{n} \log P_\theta(X_i = x_i)$$

- Why take logarithm? (Starting from the more obvious to the deepest.)
  1. Maximizing $L(\theta)$ or $l(\theta)$ yield the same optimum θ^* since $\log(x)$ is a monotonic function

2. It is easier to take derivative of a sum than a product (in case we use first-order condition to compute optimum)
3. $\log P_\theta(x)$ is often simpler when the density function $P_\theta(x)$ has an exponential form e.g. $\mathcal{N}(\mu, \sigma^2)$
4. Numerical computation of a product of many small numbers ≈ 0 results in arithmetic underflow (https://en.wikipedia.org/wiki/Arithmetic_underflow)
5. The sum of "things" has a deep connection with the $\mathcal{X}^2_k$ distribution and related hypothesis tests.

- The **maximum likelihood principle** says that the best estimate of the unknown true parameter θ is the one that makes the observed data the most likely to occur

$$\theta^*_{\text{MLE}} = \arg\max_{\theta \in \Theta} L(\theta)$$

  This optimal choice θ^*_{MLE} is the **maximum likelihood estimate MLE**.
- *Example:* For $\text{Bern}(p)$ distribution, the likelihood and log-likelihood functions are

$$L(p) = \prod_{i=1}^{n} p^{x_i}(1-p)^{1-x_i}$$

$$l(p) = \log L(p) = \sum_{i=1}^{n} x_i \log(p) + (1-x_i)\log(1-p) = \left(\sum_{i=1}^{n} x_i\right)\log(p) + \left(n - \sum_{i=1}^{n} x_i\right)\log(1-p)$$

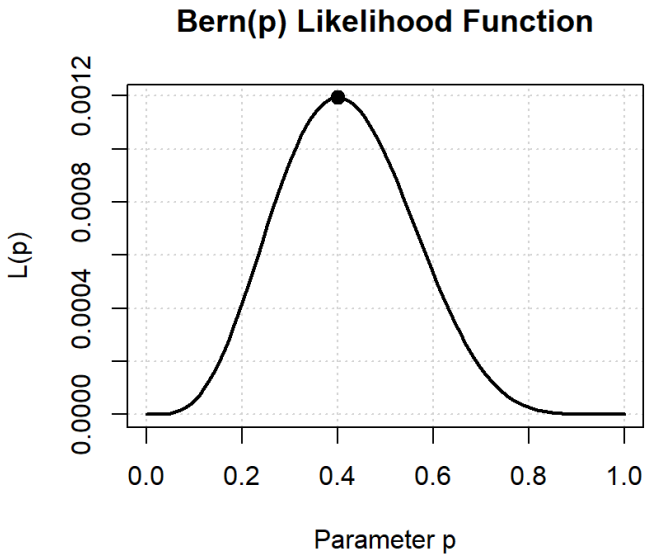
  By differentiating $l(p)$ and solving the first-order condition $l'(p) = 0$, the maximum likelihood estimate MLE is

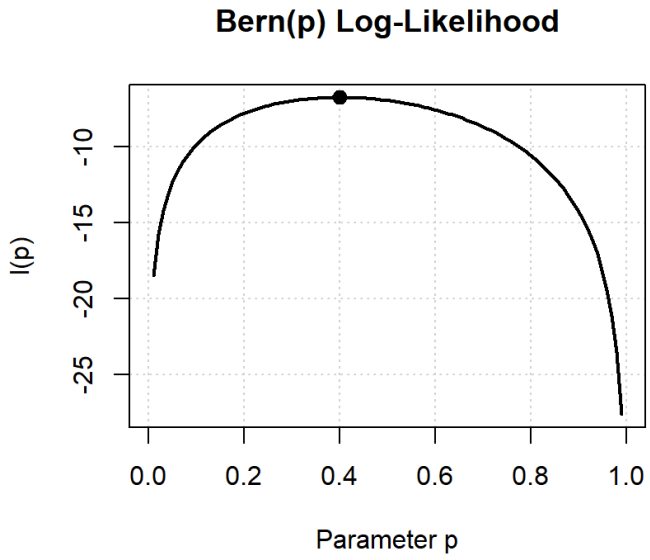
$$p^*_{\text{MLE}} = \arg\max_{p \in (0,1)} l(p) = \frac{1}{n}\sum_{i=1}^{n} x_i = \bar{x}$$

```
x <- c(0, 1, 1, 0, 0, 1, 0, 0, 0, 1)

L <- function(p) {
  return(prod(p^x * (1-p)^(1-x)))
}

l <- function(p) {
  n <- length(x)
  return(sum(x) * log(p) + (n - sum(x)) * log(1-p))
}
```





- *Example:* For $\mathcal{N}(\mu, \sigma^2)$ distribution, the likelihood and log-likelihood functions are

$$L(\mu, \sigma^2) = \prod_{i=1}^{n} f_{\mu,\sigma^2}(x_i) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{x_i-\mu}{\sigma}\right)^2}$$

$$l(\mu, \sigma^2) = \log L(\mu, \sigma^2) = \sum_{i=1}^{n} \log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{1}{2}\left(\frac{x_i-\mu}{\sigma}\right)^2 = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i-\mu)^2$$

  By differentiating $l(\mu, \sigma^2)$ over both variables μ and σ^2, and solving the two first-order conditions, the maximum likelihood estimates MLEs are

$$\mu^*_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n} x_i = \bar{x}$$

$$\sigma^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i-\bar{x})^2 = \hat{\sigma}^2$$

  In other words, they are the sample mean and (unadjusted) sample variance.
- Is the maximum likelihood principle (and hence the maximum likelihood estimate MLE) always the most ideal answer? Not necessarily.
- *Example:* For $\mathcal{N}(\mu, \sigma^2)$ distribution, the MLE σ^2_{MLE} derived directly based on the likelihood function is the (unadjusted) sample variance. It is biased.
- *Example:* For $\text{Bern}(p)$ distribution, if all $x_1 = \cdots = x_n = 0$ i.e. all n observed coin tosses are tails, then the MLE would be

$$p^*_{\text{MLE}} = \bar{x} = 0$$

  In other words, it estimates the coin is most likely to have no chance of head. This is absurd.
- In practice, the Bayesian statistical framework (https://en.wikipedia.org/wiki/Bayesian_inference) is one way to address this kind of issue.